



DOCUMENTATION TECHNIQUE

Application Web de Location de Véhicules — MACHINA

Technologie	Laravel 12 — PHP 8.4
Version	1.0.0
Date	Mars 2026
Type de projet	Application web académique / professionnel
Stack frontend	Blade + Vanilla JS + Vite (thème sombre, multilingue)
Base de données	MySQL 8 (prod) — SQLite (tests)

Sommaire

Sommaire.....	2
1. Résumé du Projet.....	3
2. Introduction.....	4
2.1 Contexte et objectifs.....	4
2.2 Périmètre fonctionnel.....	4
3. Architecture Technique.....	5
3.1 Stack technologique.....	5
3.2 Architecture MVC Laravel.....	5
3.3 Schéma de la base de données.....	6
3.4 Sécurité implémentée.....	6
3.4.1 Politique de mots de passe.....	6
3.4.2 Blocage de compte.....	6
3.4.3 Autres mesures.....	6
4. Modules Fonctionnels.....	8
4.1 Authentification.....	8
4.1.1 Connexion.....	8
4.1.2 Inscription.....	8
4.1.3 Validation du mot de passe en temps réel.....	9
4.1.4 Réinitialisation du mot de passe.....	9
4.2 Tableau de bord client.....	10
4.3 Internationalisation et thème.....	11
4.4 Catalogue de véhicules.....	12
4.5 Gestion des réservations.....	13
4.6 Gestion des documents.....	14
4.7 Inspection des véhicules.....	14
4.8 Profil client.....	15
4.9 Moyens de paiement.....	16
5. Tests et Qualité.....	17
5.1 Suites de tests PHPUnit.....	17
5.2 Procédure de tests fonctionnels.....	17
5.2.1 Installation.....	17
5.2.2 Lancement des tests.....	17
5.2.3 Scénarios manuels recommandés.....	17
5.3 Pipeline CI/CD GitHub Actions.....	18
6. Guide d'Installation.....	19
6.1 Prérequis système.....	19
6.2 Installation pas à pas.....	19
6.3 Variables d'environnement clés.....	19
7. Conformité au Cahier des Charges.....	20
8. Conclusion.....	21

1. Résumé du Projet

Machina est une application web complète de location de véhicules développée avec Laravel 12 sous PHP 8.4. Ce projet académique met en place une plateforme permettant à des clients de consulter un catalogue de véhicules, d'effectuer des réservations, de gérer leurs documents personnels et de suivre l'état de leurs locations.

L'application adopte une architecture MVC robuste, une interface moderne en thème sombre avec des accents violet/rose, et intègre des fonctionnalités d'internationalisation (français / anglais).

Livrable	Technologie	État
Application web client	Laravel 12 / Blade / Bootstrap	Réalisé
Authentification renforcée	Laravel Auth / Middleware	Réalisé
Catalogue & filtres avancés	VehicleController / Eloquent	Réalisé
Système de réservation	ReservationController	Réalisé
Gestion documentaire	DocumentController	Réalisé
Inspection départ / retour	InspectionController	Réalisé
Profil client + statistiques	ProfileController	Réalisé
Moyens de paiement (Stripe)	PaymentMethodController	En cours
Internationalisation FR / EN	Laravel i18n	Réalisé
Mode sombre / mode jour	CSS personnalisé + JS	Réalisé
Tests (7 suites PHPUnit)	PHPUnit + SQLite	Réalisé
CI/CD pipeline	GitHub Actions + Dependabot	Réalisé

2. Introduction

2.1 Contexte et objectifs

Machina est une application web de location de véhicules développée dans le cadre d'un projet académique visant à maîtriser l'ensemble du cycle de développement logiciel moderne avec Laravel 12.

L'application propose un parcours utilisateur complet : consultation d'un catalogue de véhicules, réservation en ligne avec calcul de tarifs, gestion des documents obligatoires, inspections de départ et retour, et suivi de l'historique de locations.

2.2 Périmètre fonctionnel

- Gestion complète de l'authentification avec validation renforcée des mots de passe (14 caractères minimum)
- Catalogue de véhicules avec filtres avancés : type, transmission, carburant, équipements
- Système de réservation avec gestion des tarifs jour / semaine / mois
- Gestion documentaire : permis, carte d'identité / passeport, empreinte carte bancaire
- Inspection des véhicules (départ et retour) avec capture de l'état et calcul des pénalités
- Espace profil client avec statistiques personnelles
- Gestion des moyens de paiement via Stripe (intégration en cours)
- Interface multilingue : français et anglais
- Mode jour / mode nuit commutable avec persistance

3. Architecture Technique

3.1 Stack technologique

Couche	Technologie	Rôle
Backend	PHP 8.4 / Laravel 12	Framework principal, routes, middlewares
ORM	Eloquent ORM	Modèles, relations, accesseurs
Frontend (vues)	Blade + Vanilla JS	Héritage de layout, interactions légères
Build & assets	Vite	Compilation CSS/JS, hot-reload
Styles	CSS personnalisé	Thème sombre/jour, violet/rose
Base de données	MySQL 8	Production
Tests	SQLite	Tests isolés en mémoire
Tests framework	PHPUnit	7 suites de tests
CI/CD	GitHub Actions	Build, lint, tests, audit sécurité
Dépendances	Dependabot	Mises à jour automatiques
Paieement	Stripe	Tokenisation (en cours)
Internationalisation	Laravel i18n	Français / Anglais

3.2 Architecture MVC Laravel

L'application suit le patron architectural MVC (Modèle – Vue – Contrôleur) natif à Laravel :

- **Models** : entités métier (User, Vehicle, Reservation, Document, Inspection, PaymentMethod) avec règles de validation, relations Eloquent et accesseurs
- **Views** : templates Blade avec héritage de layout. Le layout principal fournit la barre de navigation, le footer, le sélecteur de langue, le bouton mode jour/nuit et les assets compilés par Vite
- **Controllers** : un controller dédié par module fonctionnel (ReservationController, DocumentController, etc.)
- **Middlewares** : SecurityHeaders (en-têtes HTTP), Auth (protection des routes), EmailVerification, SetLocale (gestion de la langue)
- **Policies** : contrôle d'accès basé sur les rôles (RBAC) pour chaque ressource

3.3 Schéma de la base de données

users	Utilisateurs — email, password (bcrypt), rôle, statut vérification
vehicles	Parc automobile — type, marque, modèle, tarifs (jour/semaine/mois)
reservations	Réservations — dates, statut, kilométrage, pénalités
documents	Documents clients — type, statut d'approbation
inspections	Inspections départ/retour — état, photos, kilométrage
payment_methods	Moyens de paiement — type, derniers 4 chiffres, expiration
options	Options additionnelles — siège enfant, GPS, assurance
failed_login_attempts	Tentatives échouées — blocage de compte progressif

3.4 Sécurité implémentée

3.4.1 Politique de mots de passe

- Longueur minimale : 14 caractères
- Au moins 1 lettre majuscule, 1 lettre minuscule, 1 chiffre, 1 caractère spécial

Un validateur en temps réel affiche les critères manquants avec un indicateur de force (Faible / Moyen / Fort) pendant la saisie, et bloque la soumission tant que tous les critères ne sont pas remplis.

3.4.2 Blocage de compte

- Blocage automatique après 3 tentatives de connexion échouées
- Délais progressifs entre tentatives : 30 s, 45 s, puis blocage définitif
- Traçabilité via la table failed_login_attempts et notification par email

3.4.3 Autres mesures

- Middleware SecurityHeaders : Content-Security-Policy, X-Frame-Options, X-XSS-Protection, HSTS
- Hachage des mots de passe avec bcrypt — vérification d'email obligatoire avant accès
- Tokens CSRF sur tous les formulaires — requêtes paramétrées via Eloquent (anti-injection SQL)
- Audit de sécurité automatisé dans le pipeline CI/CD

4. Modules Fonctionnels

4.1 Authentification

4.1.1 Connexion

La page de connexion permet aux utilisateurs de s'identifier avec leur adresse email et leur mot de passe. Une case à cocher « Se souvenir de moi » maintient la session active. Un lien de récupération est disponible en cas d'oubli.

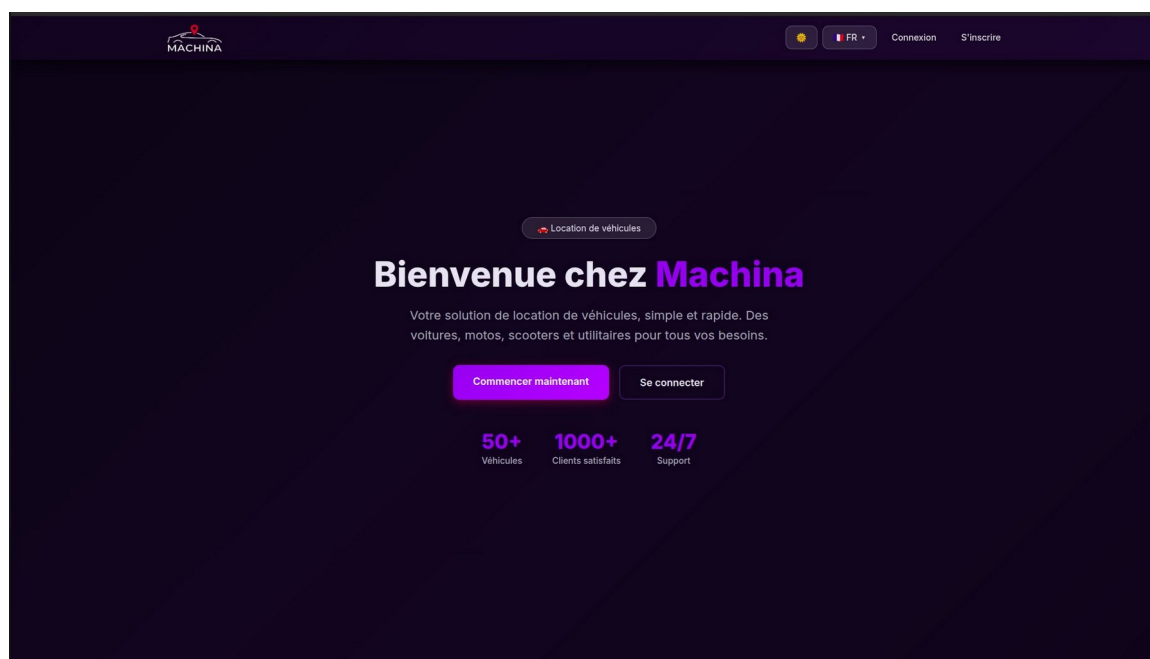


Figure 1 — Page de connexion

4.1.2 Inscription

Le formulaire d'inscription collecte les informations obligatoires : prénom, nom, email, date de naissance, téléphone, adresse (rue, code postal, ville) et mot de passe avec confirmation.

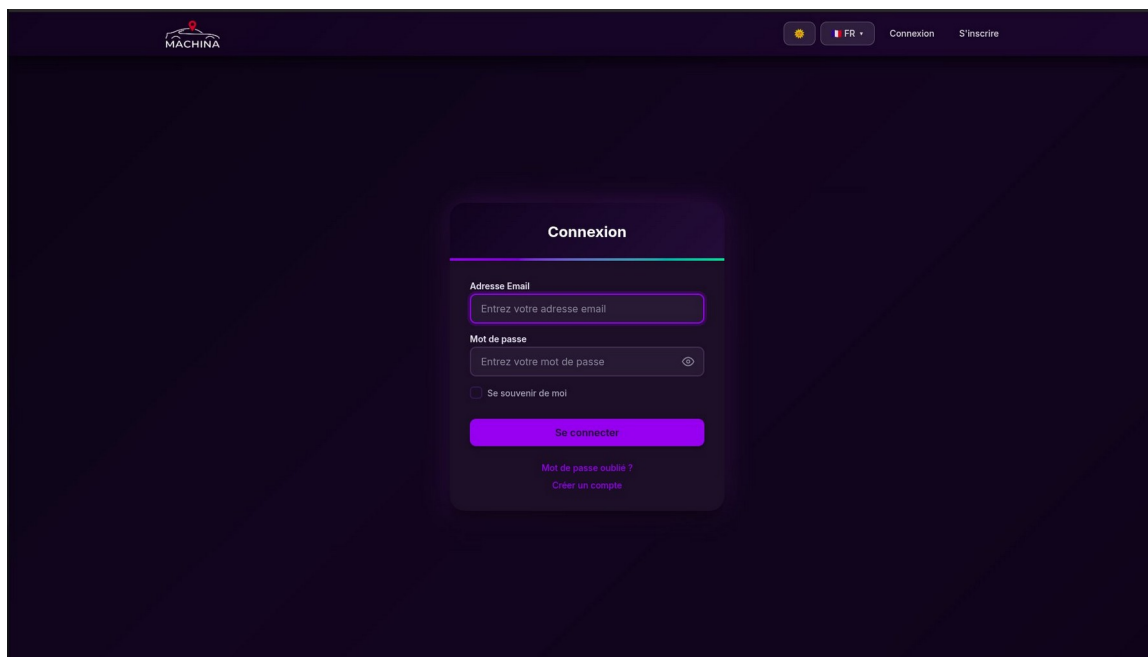


Figure 2 — Formulaire d'inscription

4.1.3 Validation du mot de passe en temps réel

Lors de la saisie, un bloc de retour immédiat liste les critères non encore satisfaits. Une barre de force colorée (Faible / Moyen / Fort) accompagne l'indicateur. La soumission est bloquée tant que tous les critères ne sont pas remplis.

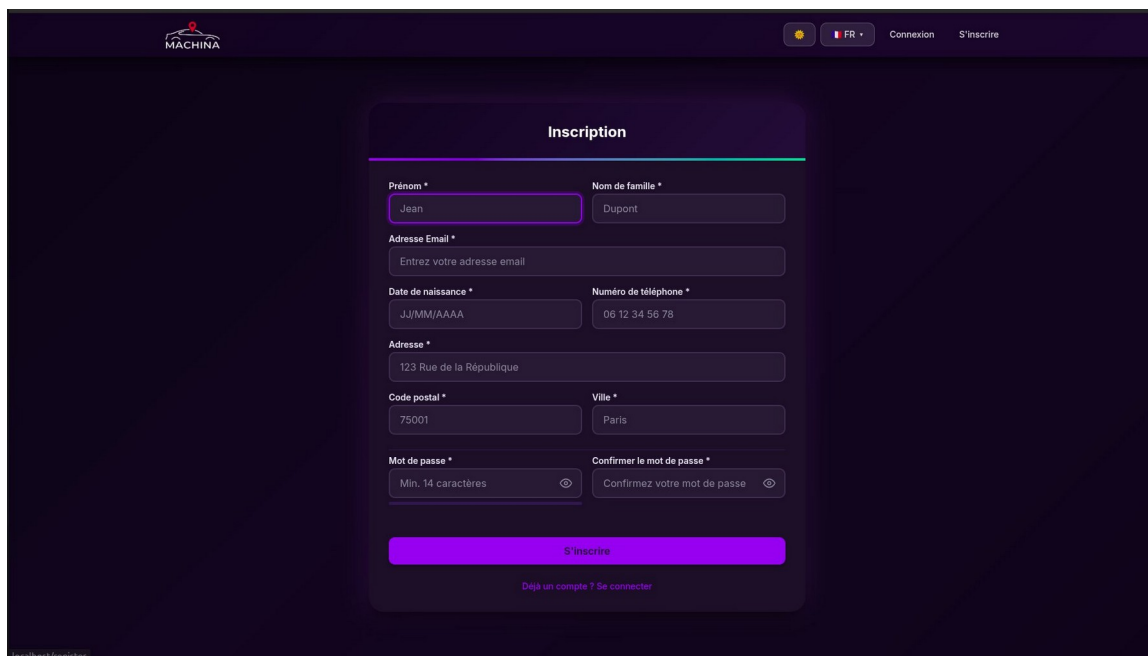
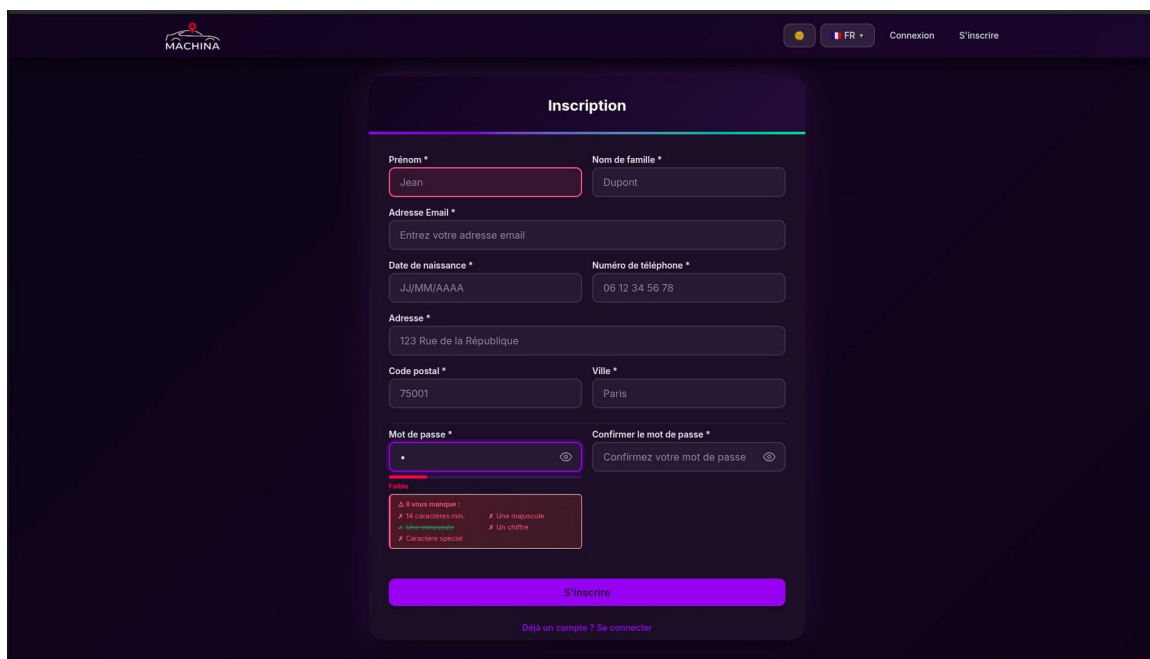


Figure 3 — Validateur de mot de passe en temps réel

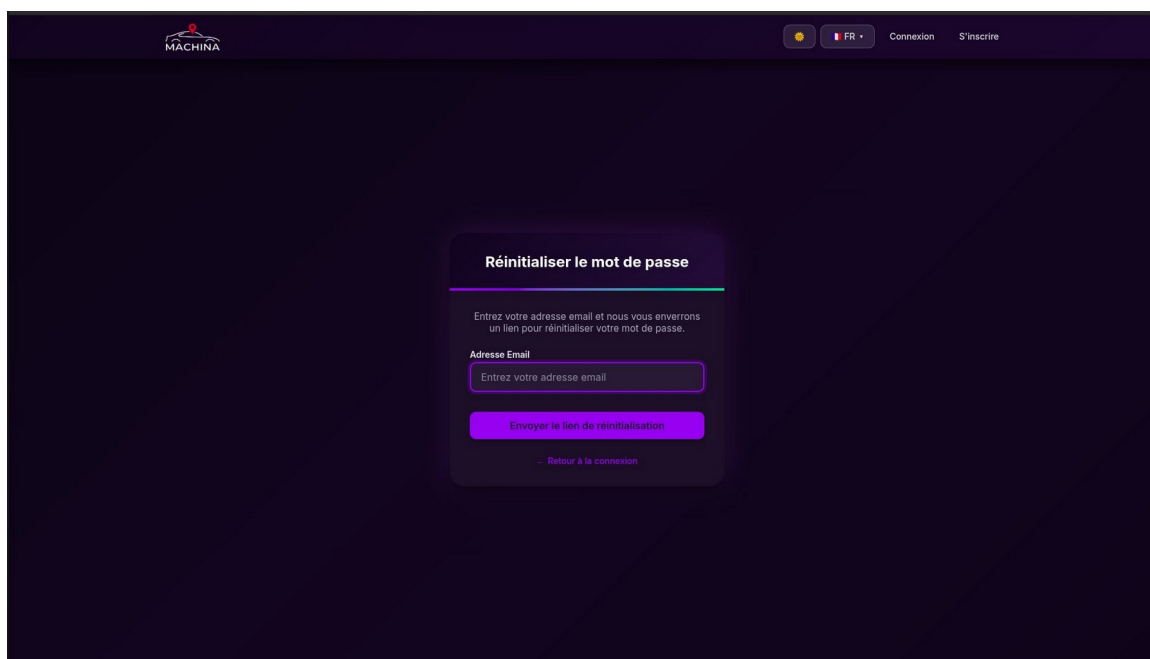
4.1.4 Réinitialisation du mot de passe

La procédure se déroule en deux étapes : (1) saisie de l'adresse email pour recevoir un lien sécurisé, (2) définition d'un nouveau mot de passe soumis aux mêmes règles de complexité.



The screenshot shows the 'Inscription' (Registration) form on the MACHINA website. The form is centered on a dark blue background. At the top, the MACHINA logo is on the left, and a language selector (FR) and links for 'Connexion' and 'S'inscrire' are on the right. The form itself has a white header with the title 'Inscription'. Below this, there are several input fields: 'Prénom *' (First Name) with the value 'Jean', 'Nom de famille *' (Last Name) with the value 'Dupont', 'Adresse Email *' (Email Address) with the placeholder 'Entrez votre adresse email', 'Date de naissance *' (Date of Birth) with the placeholder 'JJ/MM/AAAA', 'Numéro de téléphone *' (Phone Number) with the placeholder '06 12 34 56 78', 'Adresse *' (Address) with the value '123 Rue de la République', 'Code postal *' (Postal Code) with the value '75001', and 'Ville *' (City) with the value 'Paris'. Below these are two password fields: 'Mot de passe *' (Password) and 'Confirmer le mot de passe *' (Confirm Password), both with the placeholder 'Entrez votre mot de passe'. A red error message is visible below the password field: 'Le mot de passe doit contenir : 8 caractères min, Une majuscule, Un minuscule, Un chiffre, Un caractère spécial'. At the bottom of the form is a large blue button labeled 'S'inscrire' and a link 'Déjà un compte ? Se connecter'.

Figure 4 — Étape 1 : demande de réinitialisation



The screenshot shows the 'Réinitialiser le mot de passe' (Reset Password) form on the MACHINA website. The form is centered on a dark blue background. At the top, the MACHINA logo is on the left, and a language selector (FR) and links for 'Connexion' and 'S'inscrire' are on the right. The form has a white header with the title 'Réinitialiser le mot de passe'. Below this, there is a text box with the instruction: 'Entrez votre adresse email et nous vous enverrons un lien pour réinitialiser votre mot de passe.' Below the text box is an input field for 'Adresse Email' with the placeholder 'Entrez votre adresse email'. At the bottom of the form is a large blue button labeled 'Envoyer le lien de réinitialisation' and a link 'Retour à la connexion'.

Figure 5 — Étape 2 : définition du nouveau mot de passe

4.2 Tableau de bord client

Une fois connecté, le client accède à son tableau de bord personnalisé présentant des statistiques temps réel, un accès rapide aux fonctionnalités, le panneau des prochaines réservations et un journal d'activité récente.

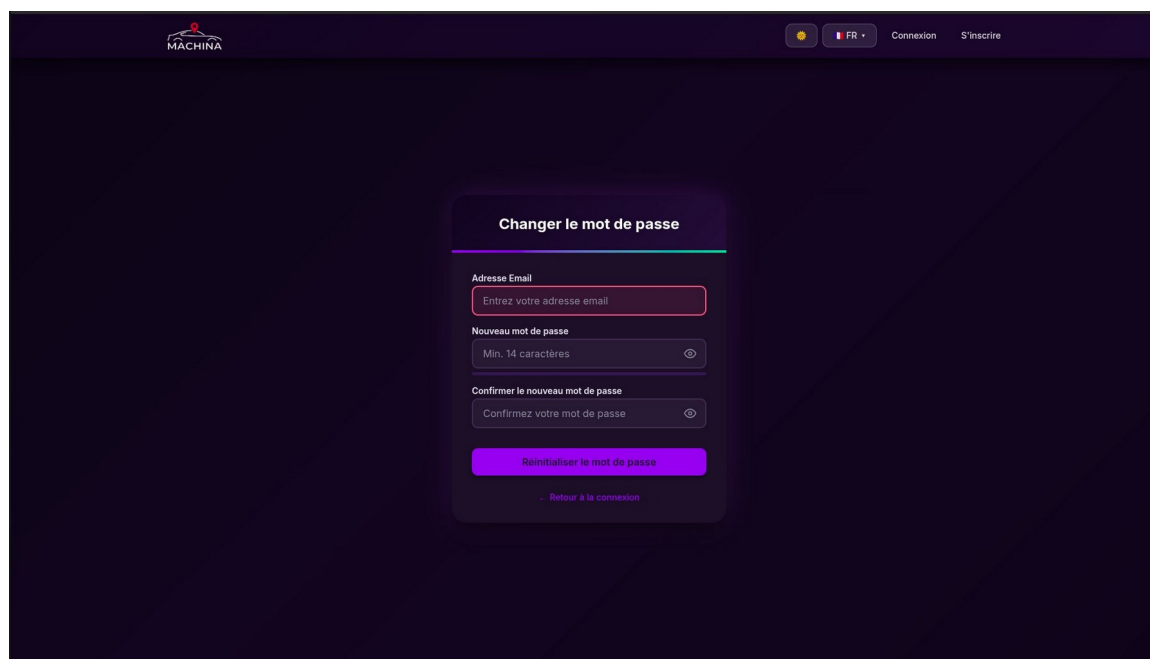


Figure 6 — Tableau de bord (mode sombre, langue française)

4.3 Internationalisation et thème

L'application propose deux fonctionnalités de personnalisation accessibles depuis la barre de navigation : un sélecteur de langue FR / EN (tous les libellés traduits) et un bouton de bascule mode sombre / mode jour.

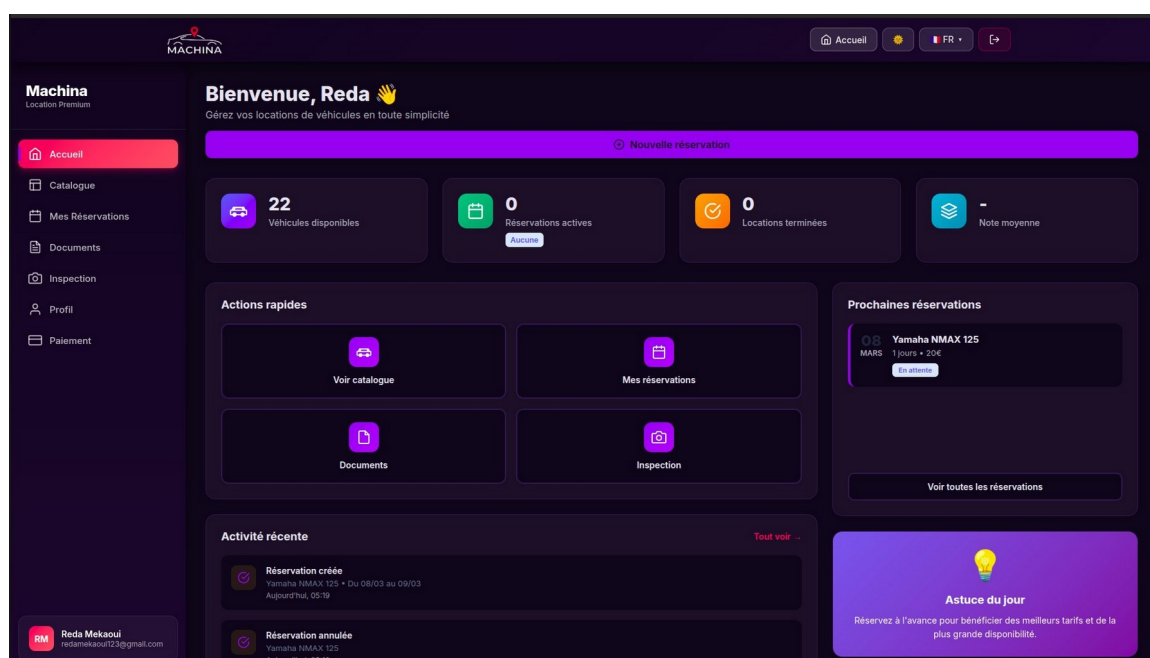


Figure 7 — Bascule mode jour/nuit et sélecteur de langue

4.4 Catalogue de véhicules

Le catalogue présente l'ensemble du parc disponible avec des filtres avancés : type de véhicule (Berline, SUV, Moto, Scooter, Camionnette, Camion), transmission, carburant et équipements. Chaque carte affiche le tarif journalier, le kilométrage et la disponibilité.

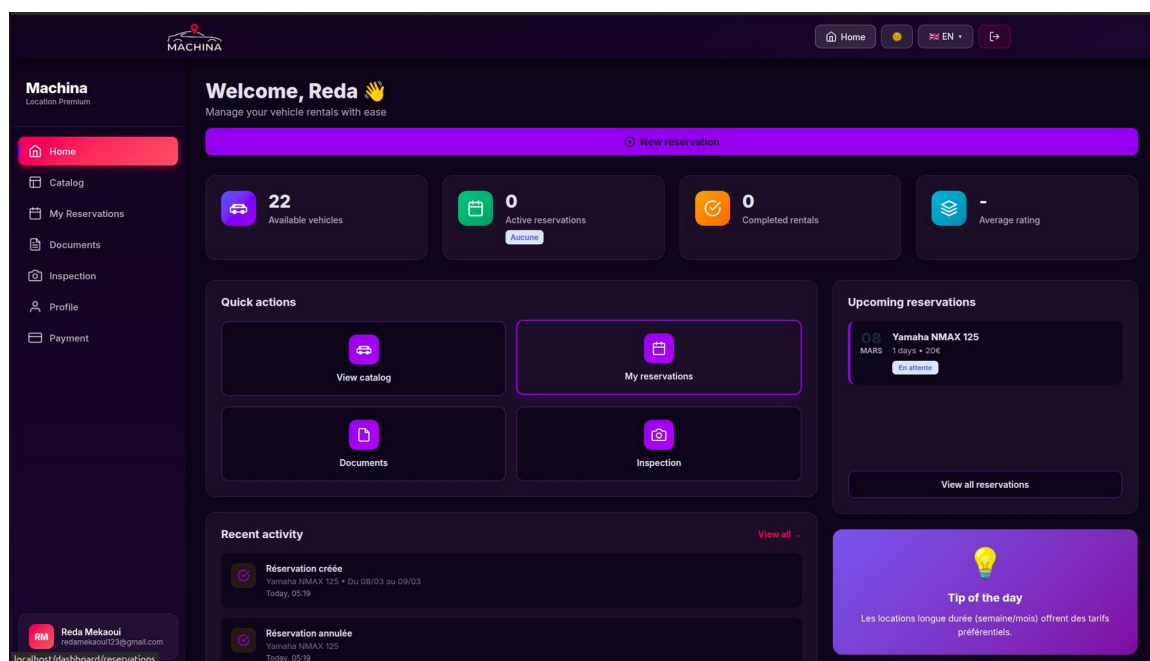


Figure 8 — Catalogue de véhicules avec filtres

4.5 Gestion des réservations

Le module de réservation permet de créer, modifier et annuler des réservations. Le système calcule automatiquement les tarifs selon la durée (jour / semaine / mois) et vérifie les conflits de disponibilité.

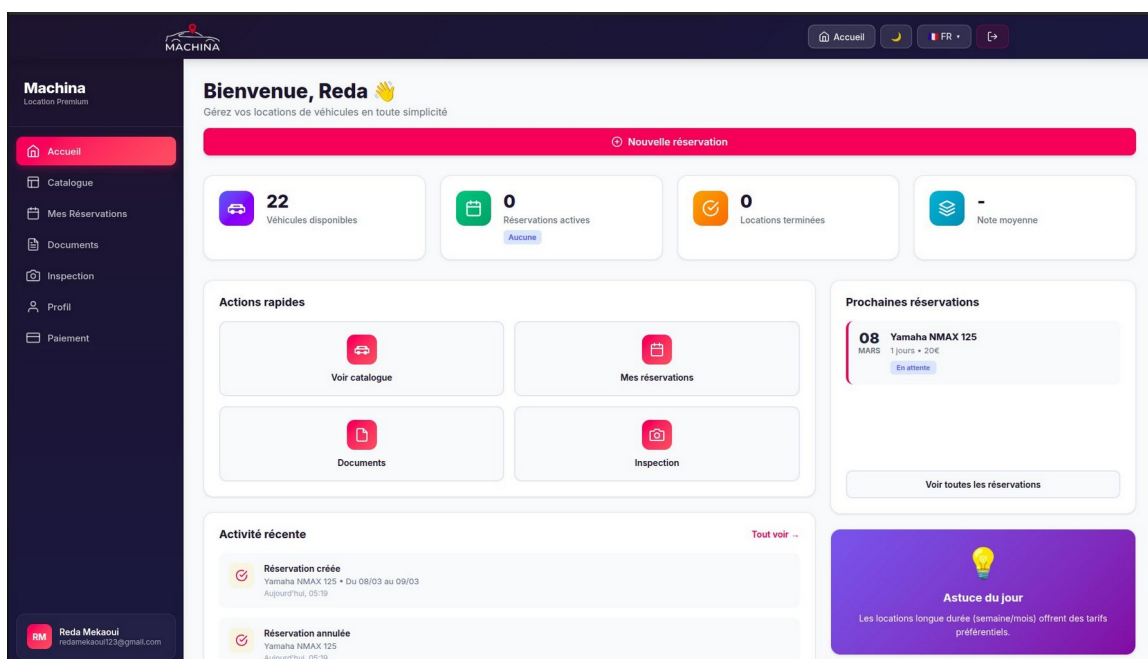


Figure 9 — Formulaire de réservation avec calcul de tarif

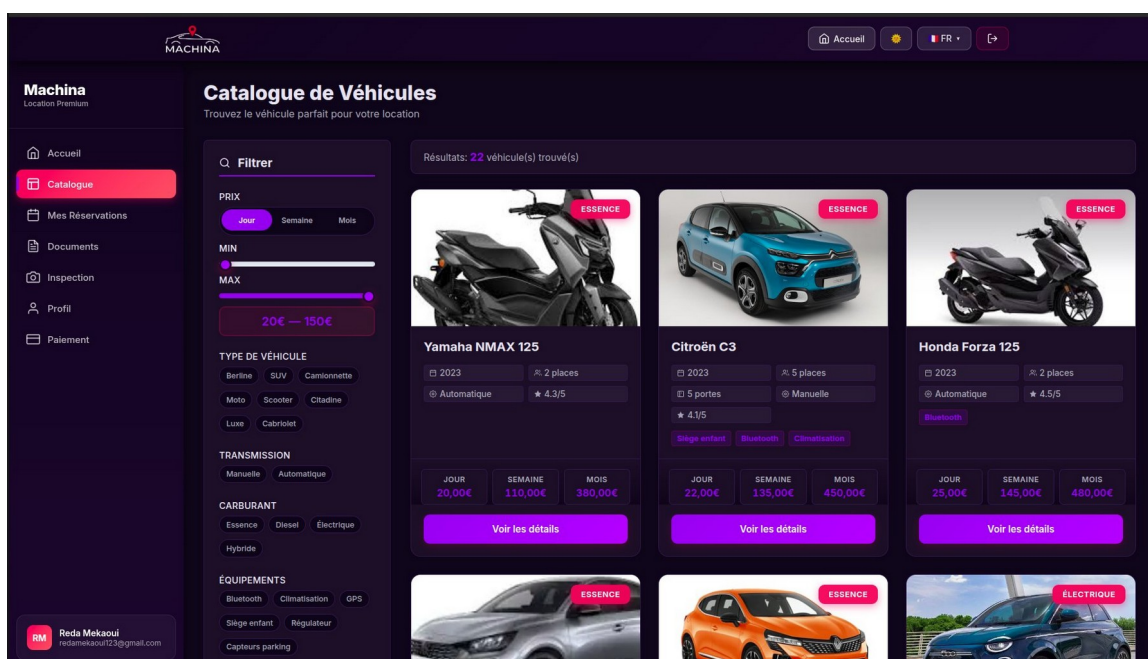


Figure 10 — Liste de mes réservations avec statuts

4.6 Gestion des documents

Trois types de documents sont obligatoires pour toute réservation : permis de conduire, carte d'identité ou passeport, et empreinte de carte bancaire. Formats acceptés : PDF, JPG, PNG (max 5 Mo). Les documents passent par un workflow d'approbation administrateur.

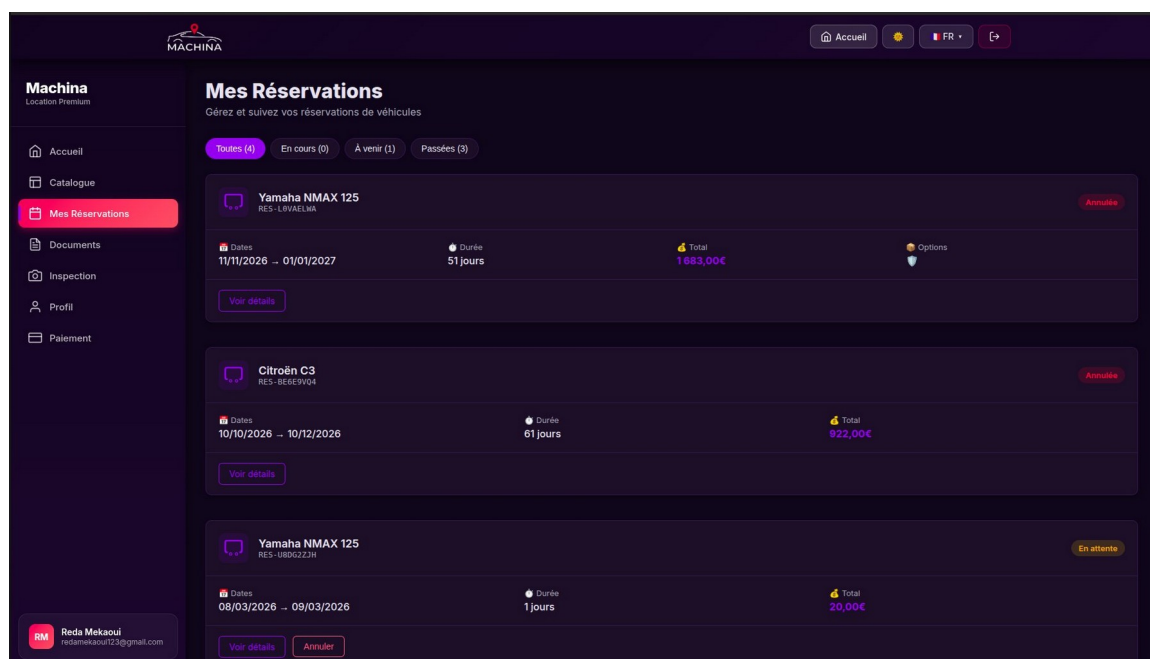


Figure 11 — Espace documents : upload et statuts

4.7 Inspection des véhicules

Le module enregistre l'état du véhicule en deux temps : inspection de départ (kilométrage initial, état, photos) et inspection de retour (kilométrage final, constat de dommages, calcul des pénalités). Ce double contrôle constitue la base contractuelle.

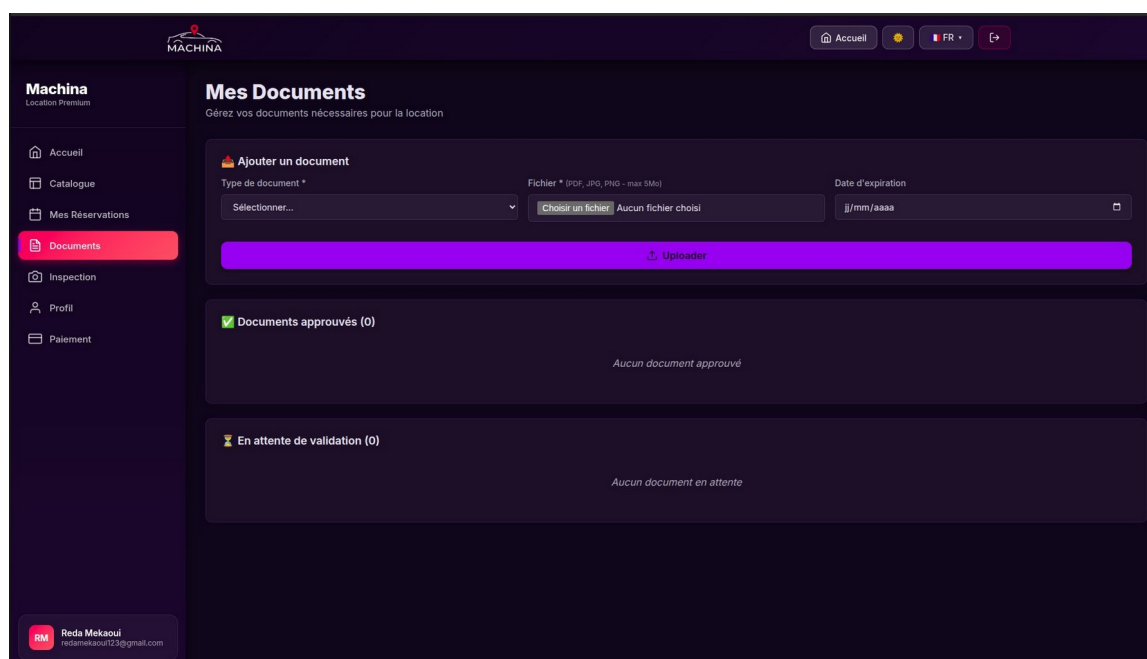


Figure 12 — Interface d'inspection départ et retour

4.8 Profil client

L'espace profil permet de consulter et modifier les informations personnelles (coordonnées, adresse) ainsi que de suivre les statistiques : nombre total de réservations, kilométrage cumulé, date d'inscription.

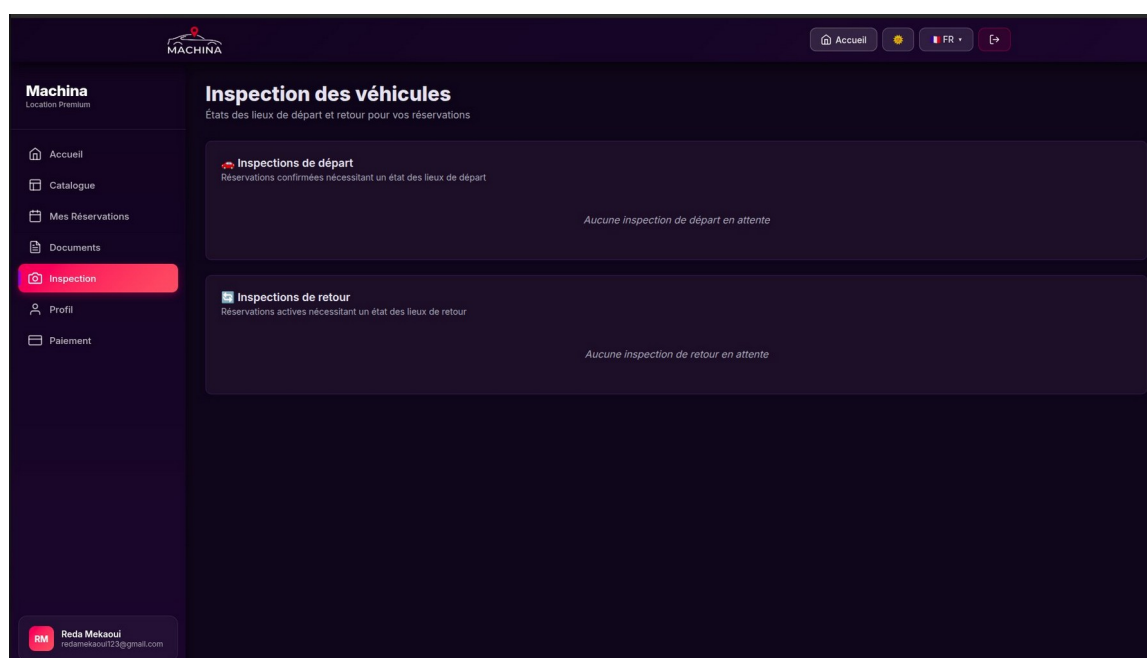


Figure 13 — Page de profil client avec statistiques

4.9 Moyens de paiement

Le module de paiement permet d'ajouter et gérer des cartes bancaires via Stripe (tokenisation sécurisée). Le formulaire collecte : nom du titulaire, numéro de carte, type (Visa, Mastercard, etc.) et date d'expiration.

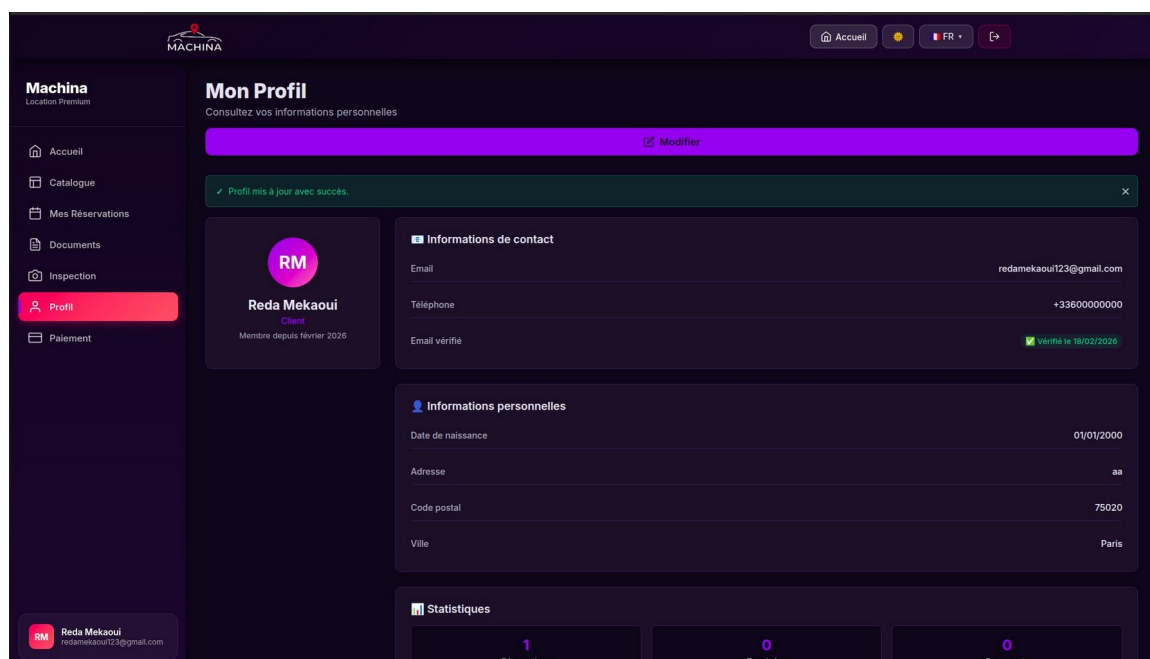


Figure 14 — Gestion des moyens de paiement

5. Tests et Qualité

5.1 Suites de tests PHPUnit

Suite	Périmètre couvert
AuthTest	Inscription, connexion, déconnexion, validation mot de passe, réinitialisation, blocage après 3 tentatives
CatalogueTest	Affichage du catalogue, filtres, pagination, disponibilité des véhicules
ReservationTest	Création, modification, annulation, calcul des tarifs, pénalités de retard
DocumentTest	Upload, validation des types/tailles, workflow d'approbation
InspectionTest	Création inspection départ/retour, calcul des dommages
ProfileTest	Lecture et mise à jour des informations profil
PaymentMethodTest	Ajout, suppression et validation des moyens de paiement

5.2 Procédure de tests fonctionnels

5.2.1 Installation

```
$ composer install
$ npm install && npm run build
$ cp .env.example .env && php artisan key:generate
$ php artisan migrate --seed
```

5.2.2 Lancement des tests

```
# Tous les tests
$ php artisan test

# Une suite spécifique
$ php artisan test --filter=AuthTest
```

5.2.3 Scénarios manuels recommandés

- Inscription : tester mot de passe trop court / sans majuscule — vérifier chaque message d'erreur

- Connexion : saisir 3 fois un mauvais mot de passe — vérifier le blocage progressif (30 s, 45 s...)
- Langue : basculer FR ↔ EN depuis la nav — vérifier que tous les libellés changent
- Thème : basculer sombre ↔ jour — vérifier la persistance après navigation
- Catalogue : utiliser chaque filtre seul puis en combinaison
- Réservation : créer une réservation valide, puis tester des dates en conflit
- Documents : uploader un PDF valide, puis un fichier >5 Mo et un format non supporté
- Inspection : compléter une inspection de départ et vérifier l'inspection de retour

5.3 Pipeline CI/CD GitHub Actions

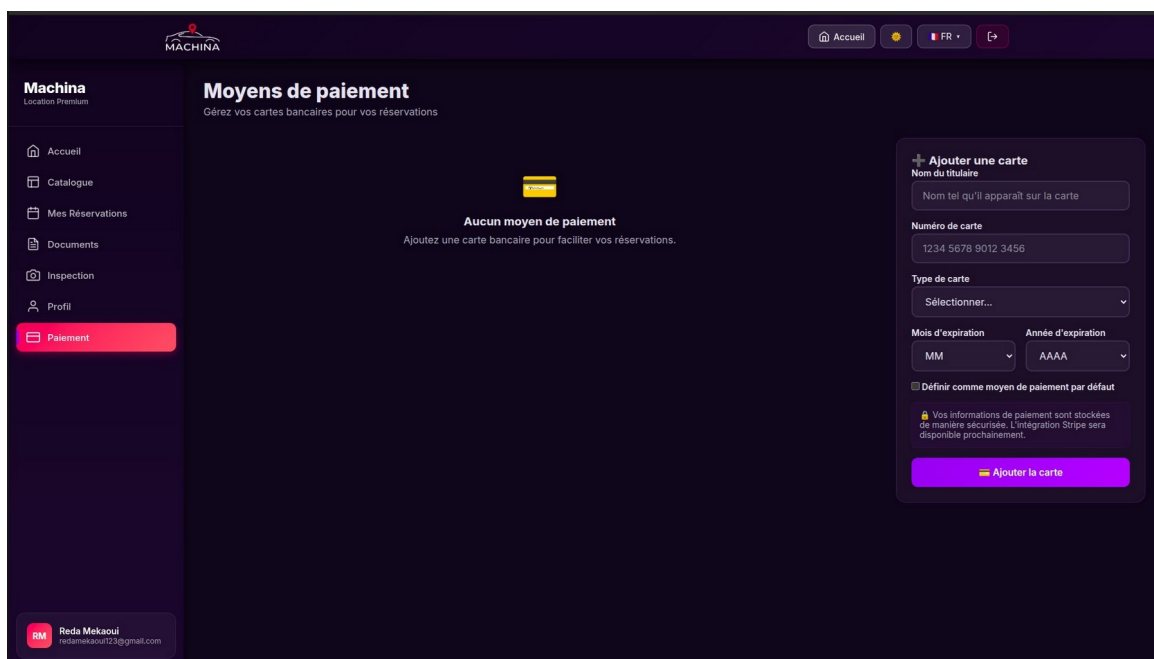


Figure 15 — Pipeline GitHub Actions

- Build et compilation des assets (Vite)
- Vérification du style de code (Laravel Pint / PHP_CS_Fixer)
- Exécution de l'ensemble des tests PHPUnit
- Audit de sécurité des dépendances (composer audit + npm audit)
- Dependabot : pull requests automatiques pour les mises à jour de dépendances

6. Guide d'Installation

6.1 Prérequis système

- PHP 8.4 avec extensions : BCMath, CType, Fileinfo, JSON, Mbstring, OpenSSL, PDO, Tokenizer, XML
- Composer 2.x — Node.js 20+ et npm 10+
- MySQL 8.0+ — Serveur web Apache, Nginx ou php artisan serve

6.2 Installation pas à pas

```
$ git clone <url-du-repo> machina && cd machina
$ composer install --no-dev --optimize-autoloader
$ npm install
$ cp .env.example .env      # éditer .env (DB, mail, Stripe)
$ php artisan key:generate
$ php artisan migrate --seed
$ npm run build
$ chmod -R 775 storage bootstrap/cache
$ php artisan serve
```

6.3 Variables d'environnement clés

APP_KEY	Clé d'application (générée par artisan key:generate)
DB_CONNECTION / DB_HOST / DB_PORT	Connexion MySQL (mysql / 127.0.0.1 / 3306)
DB_DATABASE / DB_USERNAME / DB_PASSWORD	Identifiants de la base de données
MAIL_MAILER	Driver email (SMTP, Mailgun, etc.)
STRIPE_KEY / STRIPE_SECRET	Clés Stripe pour l'intégration paiement
APP_LOCALE	Langue par défaut (fr ou en)
SESSION_LIFETIME	Durée de session en minutes (120 par défaut)

7. Conformité au Cahier des Charges

Exigence	Statut
Inscription avec validation renforcée (14 car., maj., min., chiffre, spécial)	Implémenté
Messages d'erreur détaillés par critère manquant	Implémenté
Blocage du compte après 3 tentatives échouées	Implémenté
Délais progressifs entre tentatives (30 s, 45 s, blocage définitif)	Implémenté
Types de véhicules : Berline, SUV, Moto, Scooter, Camionnette, Camion	Implémenté
Documents obligatoires : permis, empreinte CB, CNI / passeport	Implémenté
Options additionnelles : siège enfant, GPS, assurance	Implémenté
Tarifs jour / semaine / mois	Implémenté
Suivi du kilométrage	Implémenté
Inspection départ et retour avec photos	Implémenté
Pénalités de retard et de dommages	Implémenté
Internationalisation FR / EN	Implémenté
Mode jour / mode nuit	Implémenté
Intégration paiement Stripe	En cours (interface prête)
Tests unitaires et fonctionnels (7 suites PHPUnit)	Implémenté
CI/CD GitHub Actions + Dependabot	Implémenté

8. Conclusion

Le projet Machina constitue une application web complète et professionnelle de location de véhicules, développée avec les meilleures pratiques du développement web moderne. L'utilisation de Laravel 12 garantit une architecture solide, maintenable et évolutive.

Les fonctionnalités de sécurité renforcée, l'internationalisation FR/EN, la commutation de thème jour/nuit et la couverture de tests complète (7 suites PHPUnit + pipeline CI/CD) témoignent du soin apporté à la qualité du projet.

L'intégration Stripe, en cours de finalisation, complètera la chaîne fonctionnelle de bout en bout pour une mise en production prête à l'emploi.

Documentation — Mars 2026 — Projet MACHINA